Module #3 Introduction to OOPS Programming

LAB EXERCISES:

1. First C++ Program: Hello World

- Write a simple C++ program to display "Hello, World!".
- Objective: Understand the basic structure of a C++ program, including #include, main(), and cout.

Answer:

```
#include <iostream>

using namespace std;


int main() {

cout << "Hello, World!" << endl;

return 0;

}
```

2. Basic Input/Output

- Write a C++ program that accepts user input for their name and age and then displays a personalized greeting.
- Objective: Practice input/output operations using cin and cout.

Answer:

```
#include <iostream>

#include <string>

using namespace std;


int main() {

    string name;

    int age;


    cout << "Enter your name: ";

    getline(cin, name);
```

```
    cout << "Enter your age: ";
    cin >> age;

    cout << "\nHello, " << name << "!" << endl;
    cout << "You are " << age << " years old." << endl;

    return 0;
}
```

3. POP vs. OOP Comparison Program

- Write two small programs: one using Procedural Programming (POP) to calculate the area of a rectangle, and another using Object-Oriented Programming (OOP) with a class and object for the same task.
- Objective: Highlight the difference between POP and OOP approaches.

Answer:

(i) Using POP:

```
#include <iostream>
using namespace std;

int main() {
    int length, width;

    cout << "Enter length: ";
    cin >> length;

    cout << "Enter width: ";
    cin >> width;

    int area = length * width;

    cout << "Area of Rectangle = " << area;
```

```
    return 0;
}
```

(ii) Using OOP:

```
#include <iostream>
using namespace std;

class Rectangle {
public:
    int length, width;

    void getData() {
        cout << "Enter length: ";
        cin >> length;

        cout << "Enter width: ";
        cin >> width;
    }

    void displayArea() {
        cout << "Area of Rectangle = " << length * width;
    }
};

int main() {
    Rectangle r;

    r.getData();
    r.displayArea();

    return 0;
}
```

Difference Between POP and OOP

POP	OOP
Uses functions and variables directly.	Uses classes and objects.
Focuses on procedures/functions.	Focuses on data and objects.
Less secure for large programs.	Provides better data organization and security.
Suitable for small programs.	Suitable for large and complex programs.

4. Setting Up Development Environment

- Write a program that asks for two numbers and displays their sum. Ensure this is done after setting up the IDE (like Dev C++ or CodeBlocks).
- Objective: Help students understand how to install, configure, and run programs in an IDE.

Answer:

```
#include <iostream>
using namespace std;

int main() {
   int num1, num2, sum;

   cout << "Enter first number: ";
   cin >> num1;

   cout << "Enter second number: ";
   cin >> num2;

   sum = num1 + num2;

   cout << "Sum = " << sum;

   return 0;
}
```

Steps to Run the Program in an IDE

1. Install an IDE such as Dev C++ or CodeBlocks.
2. Open the IDE and create a new C++ source file.
3. Copy and paste the above code into the editor.
4. Save the file with a .cpp extension (e.g., sum.cpp).

5. Click Compile & Run (or press F9 in many IDEs).
6. Enter two numbers when prompted.
7. View the sum in the output window

THEORY EXERCISE:

1. What are the key differences between Procedural Programming and Object-Oriented Programming (OOP)?

Answer:

Procedural Programming (POP)	**Object-Oriented Programming (OOP)**
Focuses on functions and procedures.	Focuses on objects and classes.
Data and functions are separate.	Data and functions are combined into objects.
Follows a top-down approach.	Follows a bottom-up approach.
Less secure because data can be accessed easily.	More secure through data hiding and encapsulation.
Suitable for small programs.	Suitable for large and complex programs.
Examples: C, Pascal.	Examples: C++, Java, Python.

2. List and explain the main advantages of OOP over POP.

Answer:

Encapsulation (Data Security)

- OOP combines data and methods into a single unit called an object.
- Data can be protected from unauthorized access.

Code Reusability

- Through inheritance, existing classes can be reused to create new classes.
- This reduces code duplication.

Modularity

- Programs are divided into smaller classes and objects.
- Makes code easier to manage and understand.

Easy Maintenance

- Changes made in one class usually do not affect the entire program.
- Easier to update and debug large applications.

Polymorphism

- The same function can perform different tasks depending on the object.
- Improves flexibility and extensibility.

Abstraction

- Hides unnecessary implementation details from the user.
- Allows users to focus on essential features only.

Better Real-World Modelling

- Objects represent real-world entities such as students, cars, and employees.
- Makes program design more intuitive.

3. Explain the steps involved in setting up a C++ development environment.

Answer:

Steps:

1. Install a C++ Compiler

A compiler converts C++ code into an executable program.

Examples:

- GCC (MinGW)
- Clang
- MSVC (Microsoft Visual C++)

2. Install an IDE

An IDE (Integrated Development Environment) helps write, compile, and run programs easily.

Examples:

- Dev-C++
- Code::Blocks
- Visual Studio
- VS Code

3. Configure the Compiler

- Open the IDE.

- Select the installed compiler.
- Set the compiler path if needed.

4. Create a New C++ File

- Click **File** → **New**.
- Create a new source file.
- Save it with a **.cpp** extension.

5. Write the Program

Enter your C++ code in the editor.

6. Compile the Program

Click **Compile** or **Build** to check for errors and create an executable file.

7. Run the Program

Click **Run** or **Compile & Run** to execute the program.

8. Check the Output

View the output in the console window and verify the result.

4. What are the main input/output operations in C++? Provide examples.

Answer:

i. **Cin:**

- cin is used to take input from the user.

Example:

```
#include <iostream>
    using namespace std;

    int main() {
       int age;
```

```
    cout << "Enter your age: ";
    cin >> age;

    return 0;
}
```

ii. cout:

- cout is used to display output on the screen.

Example:

```
#include <iostream>
    using namespace std;

    int main() {
        cout << "Hello, World!";
        return 0;
    }
```

2. Variables, Data Types, and Operators

LAB EXERCISES:

1. Variables and Constants

- Write a C++ program that demonstrates the use of variables and constants. Create variables of different data types and perform operations on them.
- Objective: Understand the difference between variables and constants.

Answer:

```
#include <iostream>
using namespace std;

int main() {
    int age = 20;
    float salary = 25000.50;
    char grade = 'A';

    const float PI = 3.14;

    cout << "Age = " << age << endl;
    cout << "Salary = " << salary << endl;
    cout << "Grade = " << grade << endl;
    cout << "PI = " << PI << endl;

    age = 21;

    cout << "Updated Age = " << age << endl;
```

```
    return 0;
}
```

Variables: store data whose value can change during program execution.

- Examples: age, salary, grade

Constants: store fixed values that cannot be changed.

- Example: const float PI = 3.14;

2. Type Conversion

- Write a C++ program that performs both implicit and explicit type conversions and prints the results.
- Objective: Practice type casting in C++.

Answer:

```
#include <iostream>
using namespace std;

int main() {
    // Implicit Type Conversion
    int num = 10;
    double result = num;

    cout << "Implicit Conversion:" << endl;
    cout << "Integer value = " << num << endl;
    cout << "Converted to Double = " << result << endl;

    // Explicit Type Conversion
    double price = 99.99;
    int newPrice = (int)price;

    cout << "\nExplicit Conversion:" << endl;
    cout << "Double value = " << price << endl;
    cout << "Converted to Integer = " << newPrice << endl;

    return 0;
}
```

3. Operator Demonstration

- Write a C++ program that demonstrates arithmetic, relational, logical, and bitwise operators. Perform operations using each type of operator and display the results.
- Objective: Reinforce understanding of different types of operators in C++.

Answer:

```
#include <iostream>
using namespace std;

int main() {
    int a = 10, b = 5;

    // Arithmetic Operators
    cout << "Arithmetic Operators" << endl;
    cout << "a + b = " << a + b << endl;
    cout << "a - b = " << a - b << endl;
    cout << "a * b = " << a * b << endl;
    cout << "a / b = " << a / b << endl;

    // Relational Operators
    cout << "\nRelational Operators" << endl;
    cout << "a > b = " << (a > b) << endl;
    cout << "a < b = " << (a < b) << endl;
    cout << "a == b = " << (a == b) << endl;

    // Logical Operators
    cout << "\nLogical Operators" << endl;
    cout << "(a > b && b > 0) = " << (a > b && b > 0) << endl;
    cout << "(a < b || b > 0) = " << (a < b || b > 0) << endl;

    // Bitwise Operators
    cout << "\nBitwise Operators" << endl;
    cout << "a & b = " << (a & b) << endl;
    cout << "a | b = " << (a | b) << endl;
    cout << "a ^ b = " << (a ^ b) << endl;

    return 0;
}
```

- **Arithmetic Operators:** +, -, *, /
- **Relational Operators:** >, <, ==
- **Logical Operators:** && (AND), || (OR)
- **Bitwise Operators:** &, |, ^

THEORY EXERCISE:

1. What are the different data types available in C++? Explain with examples.

Answer:

Data Type	Description	Example
int	Stores whole numbers	int age = 20;

float	Stores decimal numbers	float height = 5.8f;
double	Stores large decimal numbers	double pi = 3.14159;
char	Stores a single character	char grade = 'A';
bool	Stores true or false	bool pass = true;
string	Stores text	string name = "Keyur";
void	Represents no value	void display();

2. Explain the difference between implicit and explicit type conversion in C++.

Answer:

1. Implicit Type Conversion

- Done automatically by the compiler.
- Converts a smaller data type to a larger data type.

2. Explicit Type Conversion (Type Casting)

- Done manually by the programmer.
- Used when you want to force a conversion.

3. What are the different types of operators in C++? Provide examples of each.

Answer:

Operator	**Example**
Arithmetic Operators (+, -, *, /, %)	a + b
Relational Operators (==, !=, >, <, >=, <=)	a > b
Logical Operators (&&, `	
Assignment Operators (=, +=, -=, *=, /=)	a += 5
Bitwise Operators (&, `	, ^, <<, >>`)
Increment/Decrement Operators (++, --)	a++, b--

4. Explain the purpose and use of constants and literals in C++.

Answer:

Constants

- A **constant** is a value that cannot be changed during program execution.

- **Example:**

const float PI = 3.14;

Literals

- A **literal** is a fixed value written directly in the program.
- **Examples:**

10 // Integer literal
3.14 // Floating-point literal
'A' // Character literal
"Hello" // String literal
true // Boolean literal

3. Control Flow Statements

LAB EXERCISES: 1

1. Grade Calculator
- Write a C++ program that takes a student's marks as input and calculates the grade based on if-else conditions.
- Objective: Practice conditionalstatements(if-else).

Answer:

```
#include <iostream>
using namespace std;

int main() {
   int marks;

   cout << "Enter student's marks: ";
   cin >> marks;

   if (marks >= 90)
      cout << "Grade A";
   else if (marks >= 80)
      cout << "Grade B";
   else if (marks >= 70)
      cout << "Grade C";
   else if (marks >= 60)
      cout << "Grade D";
   else
      cout << "Grade F";

   return 0;
}
```

2. Number Guessing Game

- Write a C++ program that asks the user to guess a number between 1 and 100. The program should provide hints if the guess is too high or too low. Use loops to allow the user multiple attempts.
- Objective: Understand while loops and conditional logic.

Answer:

```
#include <iostream>
using namespace std;

int main() {
   int secretNumber = 50;
   int guess;

   cout << "Guess a number between 1 and 100" << endl;

   while (true) {
      cout << "Enter your guess: ";
      cin >> guess;

      if (guess > secretNumber)
         cout << "Too High!" << endl;
      else if (guess < secretNumber)
         cout << "Too Low!" << endl;
      else {
         cout << "Congratulations! You guessed the correct number." << endl;
         break;
      }
   }

   return 0;
}
```

3. Multiplication Table

- Write a C++ program to display the multiplication table of a given number using a for loop.
- Objective: Practice using loops.

Answer:

```
#include <iostream>

using namespace std;
```

```
int main() {
    int num;

    cout << "Enter a number: ";
    cin >> num;

    for (int i = 1; i <= 10; i++) {
        cout << num << " x " << i << " = " << num * i << endl;
    }

    return 0;
}
```

4. Nested Control Structures

- Write a program that prints a right-angled triangle using stars(*) with a nested loop.
- Objective: Learn nested control structures.

Answer:

```
#include <iostream>
using namespace std;

int main() {
    int rows = 5;

    for (int i = 1; i <= rows; i++) {
        for (int j = 1; j <= i; j++) {
            cout << "* ";
        }
        cout << endl;
    }
```

```
    return 0;
}
```

THEORY EXERCISE:

1. What are conditional statements in C++? Explain the if-else and switch statements.

Answer:

- **Conditional statements** are used to make decisions in a program based on certain conditions.

1. if-else Statement

The if-else statement executes different blocks of code depending on whether a condition is true or false.

Example:

```
int age = 18;

    if (age >= 18)
        cout << "Eligible to vote";
    else
        cout << "Not eligible to vote";
```

2. switch Statement

- The switch statement is used to select one block of code from multiple options.

Example:

```
int day = 2;

    switch(day) {
        case 1:
            cout << "Monday";
            break;
        case 2:
            cout << "Tuesday";
            break;
        default:
            cout << "Invalid Day";
    }
```

2. What is the difference between for, while, and do-while loops in C++?

Answer:

Loop	Description
for	Used when the number of iterations is known.
while	Used when the number of iterations is not known. Checks the condition before execution.
do-while	Executes the loop body at least once, then checks the condition.

3. How are break and continue statements used in loops? Provide examples.\

Answer:

1. break Statement

- The break statement immediately terminates the loop.

Example**:**

```
#include <iostream>
    using namespace std;

    int main() {
       for (int i = 1; i <= 5; i++) {
          if (i == 3)
             break;
          cout << i << " ";
       }
       return 0;
    }
```

2. continue Statement

- The continue statement skips the current iteration and moves to the next iteration of the loop.

Example:

```
#include <iostream>
    using namespace std;

    int main() {
       for (int i = 1; i <= 5; i++) {
          if (i == 3)
             continue;
          cout << i << " ";
```

```
    }
    return 0;
}
```

4. Explain nested control structures with an example

Answer:

- **Nested control structures** are control statements placed inside another control statement. The most common example is a **loop inside another loop** (nested loop).

Example:

```
#include <iostream>
    using namespace std;

    int main() {
        for (int i = 1; i <= 5; i++) {
            for (int j = 1; j <= i; j++) {
                cout << "* ";
            }
            cout << endl;
        }
        return 0;
    }
```

4. Functions and scope

LAB EXERCISES:

1. Simple Calculator Using Functions

- Write a C++ program that defines functions for basic arithmetic operations (add, subtract, multiply, divide). The main function should call these based on user input.
- Objective: Practice defining and using functions in C++.

Answer:

```
#include <iostream>

using namespace std;


int add(int a, int b) {
    return a + b;
}
```

```
int subtract(int a, int b) {
    return a - b;
}

int multiply(int a, int b) {
    return a * b;
}

float divide(int a, int b) {
    return (float)a / b;
}

int main() {
    int num1, num2, choice;

    cout << "Enter two numbers: ";
    cin >> num1 >> num2;

    cout << "\n1. Add\n2. Subtract\n3. Multiply\n4. Divide\n";
    cout << "Enter your choice: ";
    cin >> choice;

    switch(choice) {
        case 1:
            cout << "Result = " << add(num1, num2);
            break;
        case 2:
            cout << "Result = " << subtract(num1, num2);
            break;
        case 3:
            cout << "Result = " << multiply(num1, num2);
            break;
        case 4:
            cout << "Result = " << divide(num1, num2);
            break;
        default:
            cout << "Invalid Choice";
    }

    return 0;
}
```

2. Factorial Calculation Using Recursion

- Write a C++ program that calculates the factorial of a number using recursion.
- Objective: Understand recursion in functions.

Answer:

```
#include <iostream>
using namespace std;

int factorial(int n) {
    if (n == 0 || n == 1)
        return 1;
    else
        return n * factorial(n - 1);
}

int main() {
    int num;

    cout << "Enter a number: ";
    cin >> num;

    cout << "Factorial = " << factorial(num);

    return 0;
}
```

4. Variable Scope

- Write a program that demonstrates the difference between local and global variables in C++. Use functions to show scope.
- Objective: Reinforce the concept of variable scope.

Answer:

```
#include <iostream>

using namespace std;


// Global variable

int num = 100;


void display() {

    // Local variable

    int num = 50;
```

```
    cout << "Local variable: " << num << endl;
    cout << "Global variable: " << ::num << endl;
}

int main() {
    display();

    cout << "Global variable in main: " << num << endl;

    return 0;
}
```

THEORY EXERCISE:

1. What is a function in C++? Explain the concept of function declaration, definition, and calling.

Answer:

- A **function** is a block of code that performs a specific task. Functions help make programs organized, reusable, and easier to understand.

1. Function Declaration

- A function declaration tells the compiler about the function's name, return type, and parameters.
- int add(int, int);

2. Function Definition

- A function definition contains the actual code that performs the task.
- int add(int a, int b) {
  return a + b;
  }

3. Function Calling

- A function is called when you want to execute it.
- int result = add(10, 20);

2. What is the scope of variables in C++? Differentiate between local and global scope.

Answer:

Local Scope

- A **local variable** is declared inside a function or block and can only be accessed within that function or block.

Example:

```
void display() {
        int x = 10;   // Local variable
        cout << x;
    }
```

Global Scope

- A **global variable** is declared outside all functions and can be accessed by any function in the program.

Example:

```
int x = 10;   // Global variable

    void display() {
        cout << x;
    }
```

Difference Between Local and Global Scope

Local Scope	Global Scope
Declared inside a function or block	Declared outside all functions
Accessible only within that function/block	Accessible throughout the program
Exists only during function execution	Exists for the entire program execution

3. Explain recursion in C++ with an example.

Answer:

- **Recursion** is a technique where a function calls itself to solve a problem. A recursive function must have a **base case** to stop the recursion.

Example:

```
#include <iostream>
    using namespace std;

    int factorial(int n) {
        if (n == 0 || n == 1)
            return 1;          // Base case
```

```
    else
        return n * factorial(n - 1);
}

int main() {
    int num = 5;

    cout << "Factorial = " << factorial(num);

    return 0;
}
```

4. What are function prototypes in C++? Why are they used?

Answer:

A function prototype is a declaration of a function that tells the compiler:

- The function's name
- The return type
- The number and types of parameters

Why are Function Prototypes Used?

1. Inform the Compiler
   - The compiler knows about the function before it is called.
2. Enable Function Calls Before Definition
   - A function can be called in main() even if its definition appears later in the program.
3. Type Checking
   - The compiler checks whether the correct number and type of arguments are passed.
4. Improve Program Organization
   - Functions can be defined separately from their usage, making code easier to read and maintain.

5. Arrays and Strings

LAB EXERCISES:

1. Array Sum and Average

- Write a C++ program that accepts an array of integers, calculates the sum and average, and displays the results.
- Objective: Understand basic array manipulation

Answer:

```
#include <iostream>

using namespace std;


int main() {
    int n, sum = 0;


    cout << "Enter the number of elements: ";

    cin >> n;


    int arr[n];


    cout << "Enter " << n << " integers:" << endl;

    for (int i = 0; i < n; i++) {

        cin >> arr[i];

        sum += arr[i];

    }


    double average = (double)sum / n;


    cout << "Sum = " << sum << endl;

    cout << "Average = " << average << endl;


    return 0;

}
```

2. Matrix Addition

- Write a C++ program to perform matrix addition on two 2x2 matrices.
- Objective: Practice multi-dimensional arrays

Answer:

```
#include <iostream>
using namespace std;

int main() {
    int matrix1[2][2], matrix2[2][2], result[2][2];

    cout << "Enter elements of first 2x2 matrix:" << endl;
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            cin >> matrix1[i][j];
        }
    }

    cout << "Enter elements of second 2x2 matrix:" << endl;
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            cin >> matrix2[i][j];
        }
    }

    // Matrix Addition
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            result[i][j] = matrix1[i][j] + matrix2[i][j];
        }
    }

    cout << "\nResultant Matrix:" << endl;
    for (int i = 0; i < 2; i++) {
```

```
        for (int j = 0; j < 2; j++) {
            cout << result[i][j] << "\t";
        }
        cout << endl;
    }

    return 0;
}
```

3. String Palindrome Check

- Write a C++ program to check if a given string is a palindrome (reads the same forwards and backwards).
- Objective: Practice string operations.

Answer:

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string str, rev = "";

    cout << "Enter a string: ";
    cin >> str;

    // Reverse the string
    for (int i = str.length() - 1; i >= 0; i--) {
        rev += str[i];
    }

    // Check palindrome
```

```
    if (str == rev) {
        cout << str << " is a palindrome." << endl;
    } else {
        cout << str << " is not a palindrome." << endl;
    }

    return 0;
}
```

THEORY EXERCISE:

1. What are arrays in C++? Explain the difference between single-dimensional and multidimensional arrays.

Answer:

- An **array** is a collection of elements of the **same data type** stored in **contiguous memory locations**. Arrays allow you to store multiple values using a single variable name.

Feature	Single-Dimensional Array	Multidimensional Array
Definition	Stores data in a single row or list.	Stores data in rows and columns (table format).
Dimensions	One dimension.	Two or more dimensions.
Declaration	int arr[5];	int arr[2][3];
Access	One index is used.	Multiple indices are used.
Example Use	Storing marks of students.	Storing a matrix or table of data.

2. Explain string handling in C++ with examples.

Answer:

- A **string** is a sequence of characters used to store and manipulate text such as names, messages, and sentences. In C++, strings are provided by the **string** class from the <string> library.

```
#include <iostream>
    #include <string>
    using namespace std;

    int main() {
       string name = "Keyur";
       cout << name;
       return 0;
    }
```

3. How are arrays initialized in C++? Provide examples of both 1D and 2D arrays.

Answer:

- **Array initialization** is the process of assigning values to array elements when the array is declared.

Syntax:

1D Array:

```
data_type array_name[size] = {values};
```

2D Array:

```
data_type array_name[rows][columns] = {{values}, {values}};
```

1. One-Dimensional (1D) Array Initialization

A 1D array stores elements in a single row.

```
#include <iostream>
    using namespace std;

    int main() {
       int numbers[5] = {10, 20, 30, 40, 50};

       for (int i = 0; i < 5; i++) {
          cout << numbers[i] << " ";
       }

       return 0;
    }
```

2. Two-Dimensional (2D) Array Initialization

A 2D array stores data in rows and columns (matrix form).

```
#include <iostream>
    using namespace std;

    int main() {
        int matrix[2][3] = {
            {1, 2, 3},
            {4, 5, 6}
        };

        for (int i = 0; i < 2; i++) {
            for (int j = 0; j < 3; j++) {
                cout << matrix[i][j] << " ";
            }
            cout << endl;
        }

        return 0;
    }
```

4. Explain string operations and functions in C++.

Answer:

- A **string** is a sequence of characters used to store text. In C++, strings are handled using the string class from the <string> library.

```
#include <string>
    using namespace std;
```

Common String Operations

1. String Initialization

```
string str1 = "Hello";
    string str2("World");
```

2. String Input and Output

```
#include <iostream>
    #include <string>
    using namespace std;

    int main() {
       string name;

       cout << "Enter your name: ";
       getline(cin, name);

       cout << "Hello, " << name;
       return 0;
    }
```

Function	Description
length()	Returns the length of the string
size()	Returns the size of the string
append()	Adds another string
substr()	Extracts a substring
find()	Finds a character or substring
erase()	Removes characters
empty()	Checks if the string is empty
clear()	Removes all characters
compare()	Compares two strings

6. Introduction to Object-Oriented Programming

LAB EXERCISES:

1. Class for a Simple Calculator

- Write a C++ program that defines a class Calculator with functions for addition, subtraction, multiplication, and division. Create objectsto use these functions.
- Objective: Introduce basic classstructure.

Answer:

```
#include <iostream>
using namespace std;

class Calculator {
public:
    // Addition
    int add(int a, int b) {
        return a + b;
    }

    // Subtraction
    int subtract(int a, int b) {
        return a - b;
    }

    // Multiplication
    int multiply(int a, int b) {
        return a * b;
    }

    // Division
    float divide(int a, int b) {
        if (b == 0) {
            cout << "Division by zero is not allowed!" << endl;
            return 0;
        }
        return (float)a / b;
```

```
    }
};

int main() {
    Calculator calc;

    int num1, num2;

    cout << "Enter two numbers: ";
    cin >> num1 >> num2;

    cout << "Addition: " << calc.add(num1, num2) << endl;
    cout << "Subtraction: " << calc.subtract(num1, num2) << endl;
    cout << "Multiplication: " << calc.multiply(num1, num2) << endl;
    cout << "Division: " << calc.divide(num1, num2) << endl;

    return 0;
}
```

2. Class for Bank Account

- Create a class BankAccount with data members like balance and member functions like deposit and withdraw. Implement encapsulation by keeping the data members private.
- Objective: Understand encapsulation in classes.

Answer:

```
#include <iostream>
using namespace std;

class BankAccount {
```

```
private:
    double balance; // Private data member

public:
    // Constructor
    BankAccount() {
        balance = 0;
    }

    // Deposit function
    void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
            cout << "Deposited: " << amount << endl;
        } else {
            cout << "Invalid deposit amount!" << endl;
        }
    }

    // Withdraw function
    void withdraw(double amount) {
        if (amount > balance) {
            cout << "Insufficient balance!" << endl;
        } else {
            balance -= amount;
            cout << "Withdrawn: " << amount << endl;
        }
    }
```

```
    // Display balance
    void displayBalance() {
        cout << "Current Balance: " << balance << endl;
    }
};

int main() {
    BankAccount account; // Create object

    account.deposit(5000);
    account.withdraw(2000);
    account.displayBalance();

    return 0;
}
```

3. Inheritance Example

- Write a program that implements inheritance using a base class Person and derived classes Student and Teacher. Demonstrate reusability through inheritance.
- Objective: Learn the concept of inheritance.

Answer:

```
#include <iostream>
using namespace std;

// Base Class
class Person {
protected:
    string name;
    int age;
```

```
public:
    void setPersonDetails(string n, int a) {
        name = n;
        age = a;
    }

    void displayPersonDetails() {
        cout << "Name: " << name << endl;
        cout << "Age: " << age << endl;
    }
};

// Derived Class: Student
class Student : public Person {
private:
    int rollNo;

public:
    void setStudentDetails(string n, int a, int r) {
        setPersonDetails(n, a);
        rollNo = r;
    }

    void displayStudentDetails() {
        displayPersonDetails();
        cout << "Roll Number: " << rollNo << endl;
    }
```

```
};

// Derived Class: Teacher
class Teacher : public Person {
private:
    string subject;

public:
    void setTeacherDetails(string n, int a, string s) {
        setPersonDetails(n, a);
        subject = s;
    }

    void displayTeacherDetails() {
        displayPersonDetails();
        cout << "Subject: " << subject << endl;
    }
};

int main() {
    Student student;
    Teacher teacher;

    student.setStudentDetails("Rahul", 20, 101);
    teacher.setTeacherDetails("Mehta", 40, "Mathematics");

    cout << "Student Details:" << endl;
    student.displayStudentDetails();
```

```
    cout << "\nTeacher Details:" << endl;
    teacher.displayTeacherDetails();


    return 0;
}
```

THEORY EXERCISE:

1. Explain the key concepts of Object-Oriented Programming (OOP).

Answer:

1. Class

A **class** is a blueprint or template for creating objects. It defines the data members and member functions that an object will have.

Example

```
class Student {
    public:
      string name;
      int age;
    };
```

Here, Student is a class.

2. Object

An **object** is an instance of a class. It is used to access the data members and functions of the class.

Example

```
Student s1;
    s1.name = "Rahul";
    s1.age = 20;
```

Here, s1 is an object of the Student class.

3. Encapsulation

Encapsulation is the process of wrapping data and functions into a single unit (class) and restricting direct access to data.

Example

```
class BankAccount {
    private:
      double balance;

    public:
      void deposit(double amount) {
        balance += amount;
      }
    };
```

4. Inheritance

Inheritance allows one class to acquire the properties and methods of another class.

Example

```
class Person {
    public:
      string name;
    };

    class Student : public Person {
    public:
      int rollNo;
    };
```

5. Polymorphism

Polymorphism means "many forms." It allows the same function or operator to behave differently in different situations.

Function Overloading Example

```
class Calculator {
    public:
      int add(int a, int b) {
        return a + b;
      }

      double add(double a, double b) {
        return a + b;
      }
    };
```

6. Abstraction

Abstraction means hiding implementation details and showing only the essential features to the user.

Example

```
class Car {
    public:
      void startCar() {
         cout << "Car Started";
      }
    };
```

2. What are classes and objects in C++? Provide an example.

Answer:

Class

A **class** is a user-defined data type that acts as a **blueprint** for creating objects. It contains data members (variables) and member functions (methods) that define the properties and behavior of an object.

Object

An **object** is an instance of a class. It is used to access the data members and member functions defined in the class.

Ex:

```
#include <iostream>

using namespace std;


// Class Definition

class Student {

public:

   string name;

   int age;
```

```
    void display() {
        cout << "Name: " << name << endl;
        cout << "Age: " << age << endl;
    }
};

int main() {
    Student s1;

    s1.name = "Rahul";
    s1.age = 20;

    s1.display();

    return 0;
}
```

3. What is inheritance in C++? Explain with an example.

Answer:

Inheritance is a feature of Object-Oriented Programming (OOP) in which a **new class (derived class)** acquires the properties and behaviors of an **existing class (base class)**.

```
#include <iostream>
using namespace std;

// Base Class
class Person {
public:
    string name;
```

```
    int age;

    void displayPerson() {
        cout << "Name: " << name << endl;
        cout << "Age: " << age << endl;
    }
};

// Derived Class
class Student : public Person {
public:
    int rollNo;

    void displayStudent() {
        displayPerson(); // inherited function
        cout << "Roll No: " << rollNo << endl;
    }
};

int main() {
    Student s1;

    s1.name = "Rahul";
    s1.age = 20;
    s1.rollNo = 101;

    s1.displayStudent();
```

```
    return 0;
}
```

4. What is encapsulation in C++? How is it achieved in classes?

Answer:

Encapsulation is one of the core concepts of Object-Oriented Programming (OOP). It means wrapping data (variables) and methods (functions) together into a single unit (class) and restricting direct access to some of the data.

In simple words:

Encapsulation = Data hiding + controlled access

Encapsulation is achieved using:

1. Classes

All data and functions are placed inside a class.

2. Access Specifiers

- private → Data cannot be accessed outside the class
- public → Data can be accessed from outside the class
- protected → Used in inheritance

Example:

```
#include <iostream>
    using namespace std;

    class BankAccount {
    private:
      double balance;  // hidden data (encapsulation)

    public:
      // Constructor
      BankAccount() {
        balance = 0;
      }
```

```
    // Public method to deposit money
    void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
        }
    }

    // Public method to withdraw money
    void withdraw(double amount) {
        if (amount <= balance) {
            balance -= amount;
        } else {
            cout << "Insufficient balance!" << endl;
        }
    }

    // Public method to view balance
    double getBalance() {
        return balance;
    }
};

int main() {
    BankAccount account;

    account.deposit(1000);
    account.withdraw(300);

    cout << "Balance: " << account.getBalance();

    return 0;
}
```